

HelpDesk Lite — Jira Delivery Workspace

Day 2 · PRD → Jira Delivery Plan (builds on the Day 1 Engineering Action Plan Pack)

This plan carries forward the Day 1 problem framing, scope boundaries, non-goals, risks, and open questions without redefining the product. Day 2 filters that work into a Jira workspace a real team could execute from — nothing here reopens product decisions that Day 1 already framed as open questions.

1. Jira Entry Filter

Every Day 1 work item is classified below before any ticket is created. Decision-type items (the unresolved questions from Day 1) are treated as their own workable tickets — "make the decision" is real work — separate from the engineering work that depends on their outcome.

Day 1 Item	Status	Reason	Next Action
Confirm V1 boundaries	Ready	No dependency; Product/Ops can confirm scope immediately.	Run a scope confirmation session; publish signed-off V1 boundary doc.
Define required request information	Ready	A decision task with no upstream dependency — it's the first thing that needs to be resolved, not something waiting on another decision.	Product/Ops draft the required-fields list; get sign-off.
Define ownership rules	Ready	Independent decision; does not require intake fields to be settled first.	Product/Ops decide manual vs. auto-assignment and team routing.
Define request states / workflow	Ready	Independent decision; can be scoped in parallel with the other two.	Product/Ops draft the request lifecycle states.
Define manager visibility requirements	Ready	Independent decision, but nothing else is blocked by it staying open a little longer.	Product/Ops list what managers need to see; defer discussion to after Sprint 1.
Decide on self-service / help content	Ready	Independent decision; lowest downstream impact of all five.	Product/Ops make a go/no-go call for V1; revisit after core epics are moving.
Build generic ticket intake skeleton	Partial	The base ticket entity (title, description, requester, timestamp) is a standard pattern that doesn't need the final field list to start.	Engineering builds the skeleton now; final custom fields wait for the decision above.
Add approved custom fields to intake	Blocked	Depends directly on "Define required request information."	Wait for field list sign-off, then implement.
Build ownership / assignment workflow	Blocked	Unlike intake, there is no safe generic skeleton here — auto vs. manual assignment changes the data model itself. Building blind risks rework.	Wait for ownership rules decision, then implement.
Build workflow / status structure	Blocked	Ticket statuses and board columns are directly defined by the request-states decision.	Wait for states decision, then configure statuses and transitions.
Build manager visibility structure (dashboards)	Blocked	Downstream of intake fields, ownership, and states all being resolved — dashboards report on data that doesn't exist yet.	Sequence after the three core epics land.
Self-service / help content build	Not Yet Ready	Not just blocked on one decision — still depends on the V1 boundary decision confirming it's even in scope. Too undefined to scope as a ticket yet.	Revisit only after the self-service go/no-go decision and V1 boundaries are both settled.
"Begin core implementation" (Day 1 umbrella item)	Not Yet Ready	Too broad to be one ticket. It's not actionable until intake fields, ownership rules, and states are all resolved — it will be broken into the scoped tickets above once that happens.	Do not create as a ticket; use it only as a milestone marker in the roadmap.

2. Delivery Structure

Hierarchy reflects real delivery layers — decisions and implementation work are split into separate stories so "Ready" work is never hidden behind a "Blocked" parent.

EPIC 1 — Request Intake & Organization
Story: Define required request information (decision) — Ready
Task: Draft required-fields list with Product/Ops
Task: Review and sign off field list
Story: Build request intake structure — Partial
Task: Build generic ticket entity/schema — Ready

- Subtask: Set up ticket entity (title, description, requester, timestamp)
- Subtask: Build basic submission form UI
- Task: Add approved custom fields — Blocked

EPIC 2 — Request Ownership

Story: Define ownership rules (decision) — Ready

Story: Build ownership / assignment workflow — Blocked

Task: Implement assignment logic per decision

Task: Add reassignment / ownership-change capability

EPIC 3 — Request Progress & Workflow

Story: Define request states / workflow (decision) — Ready

Story: Build workflow / status structure — Blocked

Task: Configure ticket statuses and transitions

Task: Build board columns matching states

EPIC 4 — Manager Visibility

Story: Define manager visibility requirements (decision) — Ready

Story: Build manager visibility structure — Blocked

Task: Build workload dashboard

Task: Build open / delayed requests view

EPIC 5 — V1 Scope & Self-Service

Story: Confirm V1 boundaries — Ready

Story: Decide on self-service / help content (decision) — Ready

Story: Self-service / help content build — Not Yet Ready

3. Ordered Backlog (First 6-8 Items)

Order is driven by two things: how much downstream work an item unblocks, and whether it's genuinely actionable today. The three core decisions rise first because everything else in Epics 1–3 waits on them. Manager visibility and self-service are independent but lower-impact, so they sit further down even though they're technically "Ready."

- Confirm V1 boundaries** — governs every other decision below it; costs almost nothing to resolve.
- Define required request information** — unblocks the highest-visibility part of the product (submission).
- Build generic ticket intake skeleton** — the one piece of real engineering that's genuinely ready today, and every other epic (ownership, states, dashboards) needs the ticket object to exist.
- Define ownership rules** — unblocks all ownership/assignment engineering.
- Define request states / workflow** — unblocks board and status configuration, which the team needs to actually run Jira day-to-day.
- Add approved custom fields to intake** — completes Epic 1 once item 2 lands.
- Build ownership / assignment workflow** — completes Epic 2 once item 4 lands.
- Configure ticket statuses and transitions** — completes Epic 3 once item 5 lands.

4. Workflow Design (Jira Statuses)

These are the statuses for the delivery board itself — the board the team uses to run this project, not the end product's ticket states (those are Epic 3's subject matter). Each status is an operational rule, not a label.

Status	What moving into it means
Backlog	Valid, filtered work (passed the entry filter above). Ordered by priority. Not yet committed to a sprint.
Ready for Sprint	Dependency or decision is resolved and the item is scoped enough that someone could start it without further clarification.
In Progress	An owner has been assigned and is actively working it. A ticket cannot enter this status without an assignee.
Blocked	Work has stopped because of an external dependency. Moving here requires a blocker reason in the ticket (comment or dedicated field) — it is never used as a silent parking spot.
In Review	The doer has finished; ownership passes to a reviewer (PM, QA, or stakeholder) for validation before acceptance.
Done	Reviewed, accepted, and meets its definition of done. No further action needed.

5. First Sprint (Sprint 1)

Sprint 1 is deliberately narrow: the three highest-impact decisions, plus the one engineering task that doesn't need any of them resolved first.

Item	Why it belongs in Sprint 1
Confirm V1 boundaries	Cheapest, highest-leverage item — sets guardrails for everything else in the sprint.
Define required request information	Unblocks the most visible and most-used part of the system first.
Define ownership rules	Unblocks the second core epic; independent of the field-list decision so it can run in parallel.
Define request states / workflow	Unblocks board configuration; also independent, so all three decisions can be worked in parallel by Product/Ops.
Build generic ticket intake skeleton	The only implementation work that is truly Ready today; keeps engineering active while decisions are being made, and every later epic depends on this entity existing.

6. Items Kept Out of Sprint 1

Item	What must happen before it can move
Add approved custom fields to intake	Waits on Sprint 1's field-list decision landing — natural Sprint 2 item.
Build ownership / assignment workflow	Waits on Sprint 1's ownership decision landing.
Build workflow / status structure	Waits on Sprint 1's states decision landing.
Define manager visibility requirements	Genuinely Ready, but resolving it doesn't unblock anything else in the sprint — deliberately deferred to protect focus, per the "don't fill Sprint 1 just because it's important" rule.
Build manager visibility structure (dashboards)	Downstream of intake, ownership, and states all being implemented — far too early.
Decide on self-service / help content	Lowest downstream impact of all decisions; deferred until core epics are moving.
Self-service / help content build	Depends on both the V1 boundary decision and the self-service decision; not yet scopeable.
"Begin core implementation" (umbrella item)	Not a real ticket — will be replaced by the scoped tasks above once Sprint 1's decisions land.

7. Visibility Controls

- **Blocked work:** moving a ticket to *Blocked* requires a blocker reason (comment or dedicated field). Blocked tickets carry a red flag on the board so they're visible at a glance instead of quietly sitting inside "In Progress."
- **Review handoffs:** moving to *In Review* automatically reassigns the ticket to the reviewer, so the board always shows who currently owns the next action — not just who last touched it.
- **Owner changes:** a ticket cannot enter *In Progress* without an assignee; every reassignment is visible on the card and in the activity log, so ownership history is never lost.
- **Priority:** a Priority field (Highest/High/Medium/Low) is shown as a colored flag on each card, and the backlog is kept manually ordered top-to-bottom as priority order, re-sorted at every grooming session.

8. Minimum Operating Layer

Dashboard view — "Delivery Health": one dashboard combining (1) all tickets currently in *Blocked* status grouped by blocker reason, (2) Sprint 1 progress, and (3) a count of decision-type tickets still open. It answers one question at a glance: is the team actually unblocking work, or just accumulating it?

Automation rule: if a ticket sits in *Blocked* status for more than 2 business days, automatically comment on the ticket tagging both its owner and the Product/Ops lead. This keeps blockers from going silent without adding process overhead anywhere else.